

Slide & Script — Phần 1-2: Giới thiệu & Cơ sở lý thuyết Web Automation Testing

Thời lượng: 12 phút / 10 slide · **Nguồn nội dung:** Report.md, mục 1 và 2

Mỗi slide gồm 3 phần: **Nội dung trên slide** (chữ hiển thị, cố giữ tối thiểu), **Hình ảnh gợi ý** (mô tả bằng lời để tìm/vẽ minh họa), và **Script thuyết trình** (lời nói đầy đủ, có thể diễn đạt lại tự nhiên khi trình bày, không cần học thuộc từng chữ).

Slide 1 — Bối cảnh & Mục tiêu Seminar

Nội dung trên slide

- Web Automation Testing — Cơ sở lý thuyết
- Manual Testing → giới hạn trước tốc độ release & regression liên tục
- Automation Testing → **Web Automation Testing** (dạng phổ biến nhất)
- Phạm vi: chỉ Web (không Mobile/API) · SUT: **EShop**

Hình ảnh gợi ý: một dòng timeline 2 bước "Manual Testing → Automation Testing" (icon người → icon robot/script), đặt ngang giữa slide; có thể thêm nhỏ logo Playwright và Cypress ở góc dưới để báo trước 2 công cụ sẽ demo ngay sau phần lý thuyết.

Script thuyết trình

Xin chào các bạn. Phần đầu tiên của seminar hôm nay, tụi mình sẽ đi qua phần giới thiệu và cơ sở lý thuyết của Web Automation Testing, để chuẩn bị nền tảng cho phần demo Playwright và Cypress mà các bạn trong nhóm sẽ trình bày ngay sau đây.

Kiểm thử phần mềm luôn đóng vai trò then chốt trong đảm bảo chất lượng. Nhưng với tốc độ phát hành nhanh và nhu cầu chạy hồi quy liên tục như hiện nay, Manual Testing — tức là tester tự tay thực hiện từng bước kiểm tra — ngày càng bộc lộ giới hạn: chậm, tốn nhân lực, khó lặp lại nhiều lần. Đó chính là lý do Automation Testing ra đời, và trong đó, Web Automation Testing là dạng phổ biến nhất, vì phần lớn ứng dụng doanh nghiệp hiện nay đều có giao diện web.

Trong seminar này, tụi mình giới hạn phạm vi chỉ ở Web Automation Testing, dùng hệ thống EShop làm SUT để demo, không đi vào Mobile hay API Testing — những mảng đó thuộc các đề tài seminar khác.

Slide 2 — Manual Testing vs Automation Testing

Nội dung trên slide

- Manual: khám phá, đánh giá UX, khó định oracle rõ ràng
- Automation: lặp lại nhất quán, rút ngắn feedback loop, chạy được trong CI/CD
- Nguyên tắc chọn: **lặp lại nhiều + expected result rõ ràng** → **Automate**
- Hai hướng **bổ trợ**, không thay thế nhau

Hình ảnh gợi ý: hai icon đối xứng hai bên slide — bên trái icon người (Manual), bên phải icon robot/script (Automation) — ở giữa là dấu "+" hoặc icon cân bằng, thể hiện tính bổ trợ.

Script thuyết trình

Trước khi đi sâu vào automation, tụi mình so sánh nhanh với Manual Testing.

Manual Testing phù hợp khi cần kiểm thử khám phá, đánh giá trải nghiệm người dùng — những thứ khó định nghĩa rõ ràng kết quả mong đợi cho máy. Nhưng nó tốn thời gian và nhân lực khi phải lặp lại nhiều lần, đặc biệt là regression, và dễ bỏ sót lỗi vì phụ thuộc sự tập trung của con người.

Automation Testing thì ngược lại: tăng khả năng lặp lại, rút ngắn thời gian phát hiện lỗi hồi quy, chạy được tự động trong CI/CD. Đổi lại là chi phí đầu tư ban đầu và chi phí bảo trì khi giao diện thay đổi.

Nguyên tắc chọn rất đơn giản: nên automate khi flow chạy lặp lại nhiều, có giá trị nghiệp vụ cao, và kết quả mong đợi rõ ràng. Còn lại, vẫn nên giữ Manual. Hai hướng này bổ trợ nhau, không thay thế hoàn toàn.

Slide 3 — Web Automation Testing & Test Pyramid

Nội dung trên slide

- Web Automation Testing = tự động hoá kiểm thử **trên trình duyệt** (khác Unit/API test)
- Test Pyramid (Martin Fowler): Unit (nhiều) → Integration (vừa) → UI/E2E (ít nhất)
- UI/E2E test: **Đễ gãy – Tốn kém – Chậm**
- Chọn lọc: ưu tiên luồng nghiệp vụ giá trị cao, không test mọi màn hình

Hình ảnh gợi ý: hình Test Pyramid kinh điển 3 tầng (tầng đáy to nhất ghi "Unit", tầng giữa "Integration/Service", tầng đỉnh nhỏ nhất "UI/E2E").

Script thuyết trình

Vậy Web Automation Testing cụ thể là gì? Đây là một nhánh của Automation Testing, thu hẹp phạm vi vào việc tự động hoá kiểm thử ngay trên trình duyệt: tự động mở trang, thao tác với giao diện, và kiểm tra kết quả hiển thị — khác với unit test kiểm tra hàm/class riêng lẻ, hay API test kiểm tra tầng service không qua giao diện.

Theo mô hình Test Pyramid của Martin Fowler, số lượng test nên giảm dần từ đáy lên đỉnh: nhiều unit test ở dưới cùng vì nhanh và rẻ, ít hơn ở tầng integration, và ít nhất là UI/E2E test ở đỉnh. Lý do là vì test qua UI có 3 nhược điểm cố hữu: dễ gãy — chỉ cần hệ thống thay đổi nhỏ là hàng loạt test fail theo; tốn kém — tốn công cụ và thời gian viết, chạy; và chậm — làm chậm pipeline.

Vì vậy, nhóm mình không cố test mọi màn hình của EShop, mà chỉ chọn ra những luồng nghiệp vụ có giá trị cao nhất, rủi ro cao nhất để automate — cụ thể luồng nào thì các bạn sẽ thấy ngay trong phần demo sắp tới.

Slide 4 — DOM & Locator

Nội dung trên slide

- DOM: cây node dựng từ HTML (id, class, data-*)

- Automation tool = truy vấn/tương tác với node trong DOM
- Locator — thang độ ổn định: **ID/Class** → **CSS/XPath** → **Role/Text/Test-id** (ổn định nhất)
- Sắp thấy: Playwright `getByRole` · Cypress `data-cy`

Hình ảnh gợi ý: bên trái vẽ mini DOM tree (vài node lồng nhau kiểu `<html>` → `<body>` → `<button id="...">`); bên phải vẽ mũi tên nằm ngang từ "kém ổn định" đến "ổn định" với 3 điểm mốc ID/Class — CSS/XPath — Role/Text/Test-id.

Script thuyết trình

Để hiểu công cụ automation làm việc như thế nào, tụi mình cần nắm 2 khái niệm nền: DOM và Locator.

DOM — Document Object Model — là cây logic mà trình duyệt dựng lên từ mã HTML, mỗi thẻ HTML tương ứng với một node mang theo các thuộc tính như id, class, hay data-*. Mọi công cụ automation, dù là Playwright, Cypress hay Selenium, đều thao tác bằng cách truy vấn và tương tác với các node trong cây DOM này.

Và Locator chính là cách xác định một node cụ thể để thao tác. Locator có một thang độ ổn định: dùng ID, Class thì nhanh nhưng dễ đổi theo style code; CSS Selector, XPath thì linh hoạt hơn nhưng dễ gây nếu bám theo vị trí lồng nhau trong DOM; còn nhóm hiện đại nhất — Role, Label, Text, Test-id — mô phỏng cách người dùng thật nhận diện giao diện, nên ít phụ thuộc cấu trúc DOM nhất.

Đây là điểm quan trọng nhất cần nhớ: lát nữa khi xem demo, các bạn sẽ thấy Playwright dùng `getByRole`, Cypress khuyến nghị thuộc tính `data-cy` — cả hai đều đang đi theo đúng xu hướng locator hiện đại này.

Slide 5 — Wait Mechanism: Chờ đúng cách

Nội dung trên slide

- Web render bất đồng bộ → chờ sai = flaky test
- 5 cấp độ: Static → Implicit → Explicit → Fluent → **Auto-Wait**
- Auto-Wait: tự chờ phần tử "actionable" (visible, enabled, stable)
- Playwright & Cypress: auto-wait tích hợp sẵn

Hình ảnh gợi ý: bậc thang 5 bước đi lên (Static Wait → Implicit Wait → Explicit Wait → Fluent Wait → Auto Wait), bậc cao nhất/cuối cùng tô màu nổi bật khác các bậc còn lại.

Script thuyết trình

Khái niệm nền thứ hai là Wait Mechanism — cơ chế chờ.

Trang web render bất đồng bộ: gọi API, chạy animation, cập nhật DOM liên tục. Nếu test không chờ đúng điều kiện mà thao tác ngay, kết quả sẽ không ổn định.

Có 5 cách chờ, từ thô sơ đến hiện đại: Static Wait là dừng một khoảng thời gian cố định — đơn giản nhưng lãng phí hoặc chưa đủ; Implicit Wait cấu hình một lần rồi tool tự chờ tối đa X giây; Explicit Wait chờ đúng một điều kiện cụ thể; Fluent Wait là biến thể tinh chỉnh được tần suất kiểm tra. Và hiện đại nhất là Auto Wait: tool tự động chờ phần tử ở trạng thái "actionable" — nghĩa là visible, enabled, ổn định — mà không cần người viết test khai báo gì thêm.

Đây chính là điểm mạnh mà cả Playwright lẫn Cypress đều quảng bá: cả hai đều có auto-wait tích hợp sẵn, đó là lý do vì sao chúng giảm được flaky test rõ rệt so với Selenium truyền thống.

Slide 6 — Assertion & Test Oracle

Nội dung trên slide

- Assertion: kiểm tra kết quả thực tế **vs** kết quả mong đợi
- Test Oracle (ISTQB): nguồn xác định expected result — **không phải chính source code**
- Lỏng quá → False Pass · Chặt quá → False Fail
- Xu hướng hiện đại: **web-first assertion** (tự retry đến khi đúng/hết giờ)

Hình ảnh gợi ý: một dòng code ngắn hiển thị to giữa slide, ví dụ `await expect(locator).toBeVisible()`, kèm chú thích nhỏ "tự động thử lại đến khi đúng hoặc hết timeout".

Script thuyết trình

Sau khi thao tác xong, làm sao biết test pass hay fail? Đó là nhờ Assertion — câu lệnh kiểm tra kết quả thực tế có khớp với kết quả mong đợi hay không.

Đứng sau assertion là khái niệm Test Oracle: theo ISTQB, oracle là nguồn để xác định kết quả mong đợi, có thể là hệ thống đối chứng, tài liệu đặc tả, hay kiến thức chuyên môn của tester — nhưng oracle không nên là chính source code đang được test.

Nếu assertion viết quá lỏng, dễ gây False Pass — bug thật bị che giấu. Nếu quá chặt, dễ gây False Fail — tốn thời gian debug test thay vì debug bug thật.

Một xu hướng hiện đại là web-first assertion: assertion tự động chờ và thử lại đến khi đúng hoặc hết thời gian, thay vì kiểm tra một lần rồi fail ngay. Ví dụ như dòng code `expect(locator).toBeVisible()` — cả Playwright và Cypress đều theo cách tiếp cận này.

Slide 7 — Flaky Test: Điểm hội tụ của mọi rủi ro

Nội dung trên slide

- Flaky Test (Google): pass/fail không nhất quán dù **code không đổi**
- 3 nguyên nhân chính: **locator dễ gãy · wait sai · vi phạm test isolation**
- Hậu quả: mất niềm tin vào cả test suite

Hình ảnh gợi ý: icon một đồng xu đang lật (hoặc xúc xắc) thể hiện kết quả "khi pass khi fail" ngẫu nhiên, đặt cạnh 3 icon nhỏ tương ứng 3 nguyên nhân (locator gãy, đồng hồ chờ, 2 test chồng chéo nhau).

Script thuyết trình

Bây giờ, tụi mình nói về vấn đề mà mọi khái niệm vừa rồi đang cố giải quyết: Flaky Test.

Theo Google Testing Blog, một flaky test là test cho kết quả không nhất quán — khi pass khi fail — dù code ứng dụng không hề thay đổi. Đây khác với test fail liên tục, vốn là tín hiệu rõ ràng về lỗi thật.

Ba nguyên nhân phổ biến nhất chính là những gì tại mình vừa học: chọn locator sai — locator dễ gãy khi UI đổi nhỏ; chờ sai — timing và network khiến thao tác chạy trước khi phần tử sẵn sàng; và vi phạm test isolation — test này ảnh hưởng đến trạng thái của test khác.

Hậu quả của flaky test rất nghiêm trọng: nó làm mất niềm tin vào cả bộ test suite. Vì vậy, việc một công cụ có auto-wait tốt, locator hiện đại, và hỗ trợ test isolation tốt — như các bạn sắp thấy ở Playwright và Cypress — chính là yếu tố quyết định chất lượng của một bộ automation test.

Slide 8 — Quy trình chung kiểm thử tự động

Nội dung trên slide

- **1. Record / Generate:** ghi lại thao tác thật để tạo script ban đầu
- **2. Chuẩn hoá script:** tách setup, testcase, assertion, teardown
- **3. Ánh xạ vào tool:** locator, action, assertion, test runner, report
- **4. Thực thi:** chạy lặp lại với nhiều bộ dữ liệu, môi trường, trình duyệt
- Mục tiêu: từ "script ghi lại được" → **test đáng tin và bảo trì được**

Hình ảnh gợi ý: flow ngang 4 bước: Record → Refactor Script → Map to Tool → Execute. Dưới mỗi bước có icon nhỏ: camera/quay màn hình, code editor, mảnh ghép/toolbox, nút play hoặc pipeline CI.

Script thuyết trình

Sau khi đã hiểu locator, wait, assertion và flaky test, tại mình có thể nhìn quy trình chung của kiểm thử tự động ở mức thực tế hơn.

Bước đầu tiên thường là Record hoặc Generate. Nghĩa là tester thao tác thật trên giao diện, còn công cụ ghi lại các hành động đó thành script ban đầu. Ví dụ như mở trang, click nút đăng nhập, nhập email, thêm sản phẩm vào giỏ hàng. Bước này giúp tạo khung nhanh, đặc biệt khi mới bắt đầu với một tool như Playwright hoặc Cypress.

Nhưng điểm quan trọng là script được record ra thường chưa nên dùng ngay trong project. Nó hay chứa locator chưa ổn định, dữ liệu hard-code, thao tác dư thừa, và thiếu cấu trúc rõ ràng. Vì vậy bước thứ hai là chỉnh lại script: thêm phần khởi tạo trước khi test, tách từng testcase độc lập, viết assertion rõ ràng, và thêm phần dọn dẹp hoặc huỷ dữ liệu sau test nếu test có tạo dữ liệu mới.

Bước thứ ba là ánh xạ logic kiểm thử vào đúng cơ chế của tool. Ví dụ trong Playwright có fixture, test runner, expect và locator theo role; trong Cypress có command chain, selector data-cy, fixture và intercept. Cùng một ý tưởng kiểm thử, nhưng mỗi tool có cách biểu diễn khác nhau.

Cuối cùng là thực thi. Test không chỉ chạy một lần với một bộ dữ liệu cố định, mà có thể chạy data-driven với nhiều input khác nhau, chạy trên nhiều trình duyệt, hoặc chạy trong CI/CD. Nếu flow phụ thuộc vào service bên ngoài, ta có thể dùng mock object hoặc mock response để cô lập test, giúp kết quả ổn định hơn.

Slide 9 — Từ script ghi lại đến test chạy ổn định

Nội dung trên slide

- **Setup:** chuẩn bị trạng thái ban đầu, user, session, dữ liệu

- **Testcase:** hành động + assertion theo expected result rõ ràng
- **Teardown:** dọn dữ liệu, reset trạng thái, tránh ảnh hưởng test sau
- **Data-driven:** một kịch bản, nhiều bộ input/expected output
- **Mock object / mock response:** cô lập dependency khó kiểm soát

Hình ảnh gợi ý: sơ đồ 3 khối dọc trong một testcase: Setup → Act & Assert → Teardown. Bên cạnh là bảng nhỏ "Test Data" với nhiều dòng dữ liệu, và một biểu tượng service bên ngoài được thay bằng "Mock".

Script thuyết trình

Slide này làm rõ hơn phần chỉnh script sau khi record. Một automated test tốt không chỉ là chuỗi thao tác click và nhập liệu. Nó nên có cấu trúc tối thiểu gồm setup, testcase chính, và teardown.

Setup là phần khởi tạo trước khi test. Ví dụ tạo user test, đăng nhập sẵn, reset giỏ hàng, hoặc đưa database về trạng thái mong muốn. Nếu không có setup rõ ràng, test sẽ phụ thuộc vào dữ liệu đang có trong hệ thống, và rất dễ bị flaky.

Phần testcase chính gồm hành động và assertion. Hành động mô phỏng flow người dùng, còn assertion kiểm tra expected result. Ví dụ sau khi thêm sản phẩm vào giỏ hàng, ta không chỉ kiểm tra đã click được nút, mà phải kiểm tra số lượng giỏ hàng tăng, sản phẩm đúng tên, đúng giá, hoặc xuất hiện thông báo thành công.

Teardown là phần dọn dẹp sau test. Nếu test vừa tạo đơn hàng, tạo tài khoản, hoặc thay đổi trạng thái sản phẩm, thì cần huỷ dữ liệu hoặc reset trạng thái để test sau không bị ảnh hưởng. Đây là một phần quan trọng của test isolation.

Khi đã có cấu trúc ổn, ta có thể mở rộng sang data-driven testing: cùng một flow đăng nhập nhưng chạy với nhiều bộ dữ liệu như tài khoản đúng, sai mật khẩu, email sai định dạng. Ngoài ra, với các dependency khó kiểm soát như cổng thanh toán, email service, hoặc API bên thứ ba, ta có thể dùng mock object hoặc mock response để giả lập phản hồi. Nhờ vậy test tập trung kiểm tra logic của hệ thống đang test, thay vì bị phụ thuộc vào môi trường bên ngoài.

Slide 10 — AI trong kiểm thử tự động

Nội dung trên slide

- 4 mức AI: **AI-assisted** · **AI-generated** · **AI-healing** · **AI-agent**
- Self-healing chỉ sửa lỗi locator do DOM đổi — không xử lý flaky do timing/dữ liệu
- Con người vẫn giữ vai trò: requirement, test oracle, review output

Hình ảnh gợi ý: 4 icon nhỏ xếp hàng ngang tương ứng 4 mức AI (bóng đèn gợi ý → cây bút tự viết → cây kim băng bó/vá → robot tự hành).

Script thuyết trình

Cuối cùng, một xu hướng không thể bỏ qua: AI trong kiểm thử tự động.

AI đang được tích hợp theo nhiều mức độ: AI-assisted — hỗ trợ gợi ý trong lúc con người vẫn chủ động viết test; AI-generated — AI sinh test script hoàn chỉnh từ mô tả yêu cầu; AI-healing — tự động sửa test khi fail do UI đổi nhỏ, hay còn gọi là self-healing locator; và AI-agent — AI tự lập kế hoạch, sinh, chạy và sửa test theo vòng lặp tự chủ.

Nhưng có một điều quan trọng cần nhớ: theo mabl, self-healing chỉ khắc phục một loại lỗi cụ thể — locator gãy do DOM đổi — chứ không xử lý được flaky do timing hay dữ liệu test. Và như James Bach từng cảnh báo về "snake oil" của các nhà cung cấp automation, con người vẫn phải là người xác định requirement, thiết kế test oracle, và review lại output của AI — không nên dùng AI như một hộp đen.

Tới đây là hết phần cơ sở lý thuyết. Những khái niệm locator, wait, assertion, flaky test mà tụi mình vừa đi qua chính là nền tảng để hiểu phần khảo sát và demo công cụ ngay sau đây.